Tensors and Automatic Differentiation

Last modified: 2026-10-25

```
# The Python chunks on this page import these; reticulate provisions them.
# Attaching reticulate gives the inline R its `py` object.
library(reticulate)
source("../tools/use-torch.R")
use_torch(c("numpy", "pandas", "matplotlib"))
```

Brian Hutchinson's CSCI 481/581 sets the order of topics in these notes, which follow it with his permission; his module on the course's tools is the starting point for this page. The exercises are new.

1 Welcome

1.1 Plan for this lecture

All code in these notes, and in the homework, uses PyTorch [1]. This lecture covers the two parts of PyTorch that everything else is built on, on the handwritten digits from the course overview:

1. how PyTorch stores the arrays that hold a network's inputs, parameters[1] and outputs;
2. how PyTorch computes the derivative of a loss from the code that computes the loss.

2 Tensors

2.1 The digits in PyTorch

Exercise 2.1 (The digits in PyTorch)**.** Load the digits from `data/optdigits-test.csv`[a], the local copy of the UCI data[b] [2] that the course overview uses. Put their pixels into one PyTorch array that holds each image as an 8 by 8 grid, and their labels into another.

[a] https://morrison-lab.github.io/dds/chapters/data/optdigits-test.csv
[b] https://archive.ics.uci.edu/dataset/80

Solution

Solution 2.1.

[1] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-parameter

```
import numpy as np
import pandas as pd
import torch
import matplotlib.pyplot as plt

digits = pd.read_csv("data/optdigits-test.csv", header=None)
pixels = digits.iloc[:, :64].to_numpy()
images = torch.tensor(pixels.reshape(-1, 8, 8))
labels = torch.tensor(digits[64].to_numpy())
n_images = len(images)
print(images[0])
```

```
tensor([[ 0,  0,  5, 13,  9,  1,  0,  0],
        [ 0,  0, 13, 15, 10, 15,  5,  0],
        [ 0,  3, 15,  2,  0, 11,  8,  0],
        [ 0,  4, 12,  0,  0,  8,  8,  0],
        [ 0,  5,  8,  0,  0,  9,  8,  0],
        [ 0,  4, 11,  0,  1, 12,  7,  0],
        [ 0,  2, 14,  5, 10, 12,  0,  0],
        [ 0,  0,  6, 13, 10,  0,  0,  0]])
```

```
print(labels[:10])
```

```
tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

`torch.tensor` copies a NumPy array into a PyTorch array. `images` stacks 1797 grids, each with 8 rows and 8 columns, and `labels` holds one number per image.

Definition 2.1 (Tensor)**.** A **tensor** is an array of numbers arranged on a regular grid with any number of dimensions [3, Sec. 2.1]. Every entry is picked out by one index per dimension.
A tensor with no dimensions is a scalar, one with one dimension is a vector, and one with two is a matrix. PyTorch says "dimensions" (`ndim`, and `dim=` in its functions), where NumPy, which also says `ndim`, calls them "axes" (`axis=`); other sources say "modes", or call the number of dimensions the tensor's "order" or "rank". That "rank" is not the rank of a matrix. Physics and mathematics use "tensor" for a multilinear map, which an array of numbers represents once a basis is chosen; deep learning means only the array.

Example 2.1 (The digits at three sizes)**.**

```
print(images.ndim, images[0].ndim, images[0, 3].ndim, images[0, 3, 4].ndim)
```

```
3 2 1 0
```

```
print(images[0, 3])
```

```
tensor([ 0,  4, 12,  0,  0,  8,  8,  0])
```

```
print(images[0, 3, 4])
```

```
tensor(0)
```

`images` from Solution 2.1 has three dimensions: image, row and column. Indexing it once, as `images[0]`, gives the first image, a matrix; indexing twice, as `images[0, 3]`, gives that image's row 3, a vector of 8 pixels; and indexing three times, as `images[0, 3, 4]`, gives one pixel, a scalar, with rows and columns counted from 0.

Definition 2.2 (Shape)**.** A tensor's **shape** lists the size of each of its dimensions, in order. PyTorch reports it as `.shape`, or `.size()`; NumPy also calls it `.shape`.

Example 2.2 (The shapes of the digits)**.**

```
print(images.shape, images[0].shape, images[0, 3].shape, images[0, 3, 4].shape)
```

```
torch.Size([1797, 8, 8]) torch.Size([8, 8]) torch.Size([8]) torch.Size([])
```

`images` has shape (1797, 8, 8): 1797 images, each with 8 rows and 8 columns. Indexing it drops the dimensions that the index fixes: one image has shape (8, 8), one of its rows has shape (8,), and one pixel has the empty shape ().

2.2 The average pixel

Exercise 2.2 (The average pixel)**.** Find the average of the 64 pixels of the first image with the tensor method `.mean()`.

Solution

Solution 2.2.

```
import textwrap

print(images.dtype)
```

```
torch.int64
```

```
try:
    images[0].mean()
except RuntimeError as err:
    print(textwrap.fill(str(err), 60))
```

```
mean(): could not infer output dtype. Input dtype must be
either a floating point or complex dtype. Got: Long
```

```
first_mean = images[0].float().mean().item()
print(images[0].float().dtype, first_mean)
```

```
torch.float32 4.59375
```

The pixels were read as whole numbers, so the tensor stores 64-bit integers, `torch.int64`, which PyTorch also calls `Long`. `.mean()` refuses them, since an average of integers is not in general an integer. Converting the tensor with `.float()` to 32-bit floating-point numbers fixes it: the first image's average pixel is 4.59, on the file's scale of 0 to 16.

Definition 2.3 (Data type)**.** A tensor's **data type** (in PyTorch, its `dtype`) is the kind of number that every one of its entries stores, such as 64-bit integers (`torch.int64`) or 32-bit floating-point numbers (`torch.float32`, PyTorch's default for real numbers). All the entries of one tensor share one data type. PyTorch's tensor attributes[a] page lists them all.

[a] https://docs.pytorch.org/docs/stable/tensor_attributes.html

Example 2.3 (Two data types for one image)**.**

```
pixel_int = images[0, 3, 4]
pixel_float = pixel_int.float()
print(pixel_int, pixel_int.dtype)
```

```
tensor(0) torch.int64
```

```
print(pixel_float, pixel_float.dtype)
```

```
tensor(0.) torch.float32
```

```
print(pixel_int == pixel_float)
```

```
tensor(True)
```

In Solution 2.2, the first image is `torch.int64` as loaded and `torch.float32` after `.float()`. One of its pixels, printed both ways, shows the difference: the float prints with a decimal point, the two compare as equal, and only the data type that stores the value differs.

2.3 Reshaping and averaging

Exercise 2.3 (Back to rows of 64)**.** Turn `images` into a tensor with one row of 64 pixels per image, the arrangement the file stores.

Solution

Solution 2.3.

```
rows = images.reshape(-1, 64)
print(rows.shape, torch.equal(rows, torch.tensor(pixels)))
```

```
torch.Size([1797, 64]) True
```

`reshape(-1, 64)` asks for rows of 64 and lets PyTorch work out the number of rows, the `-1`. It reads the entries in order, last dimension fastest, so each image's row 0 comes first, then its row 1, and so on, which undoes the `reshape(-1, 8, 8)` of Solution 2.1. `torch.equal` confirms that the shapes and every entry match. The course overview's remark on the two arrangements says which models take which.

Exercise 2.4 (The average image of each digit)**.** Compute the average image of each digit, 0 to 9, as one tensor of shape (10, 8, 8).

Solution

Solution 2.4.

```
averages = torch.stack(
    [images[labels == k].float().mean(dim=0) for k in range(10)]
)
print(averages.shape)
```

```
torch.Size([10, 8, 8])
```

`labels == k` is a tensor of `True` and `False`, one per image, and indexing `images` with it keeps the images of digit `k`. `.mean(dim=0)` averages over dimension 0, the images, and leaves one average for each row and column. `torch.stack` puts the ten 8 by 8 averages along a new first dimension.

Exercise 2.5 (Drawing the averages)**.** Draw the ten averages from Exercise 2.4 side by side.

Solution

Solution 2.5.

```
def draw_digits(grids):
    """Draw each 8 by 8 grid in `grids` side by side, titled by its index."""
    fig, axes = plt.subplots(1, len(grids), figsize=(8, 1.2), layout="constrained")
    for k, ax in enumerate(axes):
        ax.imshow(grids[k], cmap="gray_r", vmin=0, vmax=16)
        ax.set_title(str(k))
        ax.set_axis_off()
    plt.show()

draw_digits(averages)
```

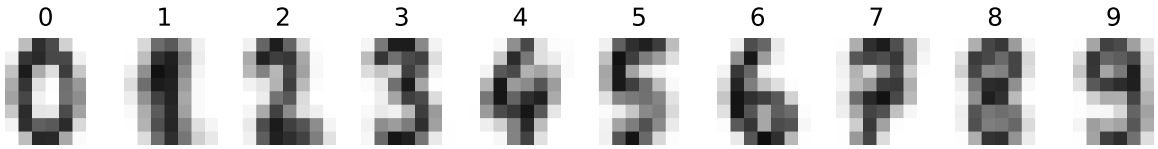


Figure 1: The average image of each digit among the 1797 images.

`imshow` draws each 8 by 8 average as a grid of gray squares, on the file's scale of 0 (white) to 16 (black).

2.4 Tensors and NumPy arrays

Exercise 2.6 (A tensor that shares its array)**.** `torch.from_numpy`[a] also makes a tensor from a NumPy array. Make one this way from a copy of `pixels`, change an entry of the tensor, and look at the same entry of the array.

[a] https://docs.pytorch.org/docs/stable/generated/torch.from_numpy.html

Solution

Solution 2.6.

```
shared_array = pixels.copy()
shared = torch.from_numpy(shared_array)
shared[0, 0] = 99
print(pixels[0, 0], shared_array[0, 0])
```

```
0 99
```

The tensor from `torch.from_numpy` uses the array's own memory, so changing the tensor changed the array, from 0 to 99. The tensor method `.numpy()` shares memory the same way, in the other direction.

Exercise 2.7 (A tensor that copies its array)**.** Repeat Exercise 2.6 with `torch.tensor` in place of `torch.from_numpy`.

Solution

Solution 2.7.

```
copied_array = pixels.copy()
copied = torch.tensor(copied_array)
copied[0, 0] = 99
copied_entry = copied[0, 0].item()
print(copied_array[0, 0], copied_entry)
```

```
0 99
```

`torch.tensor` copies the numbers, so the array kept 0 while the tensor changed to 99. Sharing, as in Solution 2.6, saves time and memory on large data, at the cost of a change in one place showing up in the other.

2.5 Devices

Definition 2.4 (Device)**.** A tensor's **device** is the processor whose memory holds its entries, and on which operations on the tensor run: the CPU, or a GPU (in PyTorch, `"cuda"` for an NVIDIA GPU). PyTorch makes a tensor on the CPU unless told otherwise, `.to()` copies a tensor to another device, and an operation on two tensors needs both on the same device.

Example 2.4 (Where the digits live)**.**

```
has_gpu = torch.cuda.is_available()
print(images.device, has_gpu)
```

```
cpu False
```

`images` was made without naming a device, so it lives on the CPU. `torch.cuda.is_available()` says whether this computer has an NVIDIA GPU that PyTorch can use; it returned `False` on the computer that built this page. Where it returns `True`, `images.to("cuda")` makes a copy of `images` on the GPU.

Read

PyTorch's tensors tutorial[a] shows more ways to make, index and combine tensors. Its notes on broadcasting[b] give the rules for an operation on two tensors of different shapes.

[a] https://docs.pytorch.org/tutorials/beginner/basics/tensorqs_tutorial.html
[b] https://docs.pytorch.org/docs/stable/notes/broadcasting.html

3 Automatic differentiation

3.1 The soft 0-or-1 rule

The rest of this page differentiates the soft 0-or-1 rule of the machine learning review. Among the images of 0s and 1s, it gives each image the probability $\hat{\pi} = \text{expit}(x - \theta)$ of being a 1, where x is the image's center ink, its average pixel over rows 2 to 5 and columns 3 and 4, and θ is the rule's cutoff. Its mean log loss[2] over those n images, its empirical risk[3] under that loss, is

$$\hat{R}(\theta) = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log \hat{\pi}_i + (1 - y_i)\log(1 - \hat{\pi}_i)],$$

where y_i is 1 if image i is a 1 and 0 if it is a 0.

Exercise 3.1 (The data for the soft rule)**.** From `images` and `labels`, make a tensor of the center ink x_i of each image of a 0 or a 1, stored as 64-bit floating-point numbers, and a tensor of the matching y_i.

Solution

Solution 3.1.

[2] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-log-loss
[3] https://morrison-lab.github.io/lds/chapters/big-ideas.html#def-empirical-risk

```
center_ink = images[:, 2:6, 3:5].double().mean(dim=(1, 2))
is_01 = (labels == 0) | (labels == 1)
ink_01 = center_ink[is_01]
is_one = (labels[is_01] == 1).double()
n_01 = len(ink_01)
print(n_01)
```

```
360
```

```
print(ink_01[:5])
```

```
tensor([ 0.3750, 15.8750,  3.0000, 14.1250,  2.5000], dtype=torch.float64)
```

```
print(is_one[:5])
```

```
tensor([0., 1., 0., 1., 0.], dtype=torch.float64)
```

`images[:, 2:6, 3:5]` keeps rows 2 to 5 and columns 3 and 4 of every image, and `.mean(dim=(1, 2))` averages over those rows and columns, leaving one number per image. `.double()` converts to 64-bit floats, `torch.float64`, so that later comparisons of derivatives are not limited by 32-bit rounding. There are $n = 360$ images of 0s and 1s.

3.2 The slope by hand

Exercise 3.2 (The slope of the log loss)**.** Find the derivative $\frac{d\hat{R}}{d\theta}$ of the mean log loss[a] with respect to the cutoff.

Hint

The logistic function[b] has derivative $\text{expit}'(z) = \text{expit}(z)(1 - \text{expit}(z))$ (lds[c]).

[a] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-log-loss
[b] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#exm-logistic-function
[c] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#thm-logistic-derivative

Solution

Solution 3.2. Write ℓ_i for image i's log loss[a], $\ell_i = -y_i \log \hat{\pi}_i - (1 - y_i) \log(1 - \hat{\pi}_i)$. Its derivative with respect to $\hat{\pi}_i$ is

$$\frac{d\ell_i}{d\hat{\pi}_i} = -\frac{y_i}{\hat{\pi}_i} + \frac{1 - y_i}{1 - \hat{\pi}_i}.$$

Since $\hat{\pi}_i = \text{expit}(x_i - \theta)$, the chain rule and the derivative of the logistic function give

$$\begin{aligned} \frac{d\hat{\pi}_i}{d\theta} &= \text{expit}'(x_i - \theta) \cdot \frac{d}{d\theta}(x_i - \theta) \\ &= \hat{\pi}_i(1 - \hat{\pi}_i) \cdot (-1) \\ &= -\hat{\pi}_i(1 - \hat{\pi}_i). \end{aligned}$$

Multiplying the two,

$$\begin{aligned} \frac{d\ell_i}{d\theta} &= \frac{d\ell_i}{d\hat{\pi}_i} \cdot \frac{d\hat{\pi}_i}{d\theta} \\ &= \left(-\frac{y_i}{\hat{\pi}_i} + \frac{1 - y_i}{1 - \hat{\pi}_i} \right) \cdot (-\hat{\pi}_i(1 - \hat{\pi}_i)) \\ &= y_i(1 - \hat{\pi}_i) - (1 - y_i)\hat{\pi}_i \\ &= y_i - y_i\hat{\pi}_i - \hat{\pi}_i + y_i\hat{\pi}_i \\ &= y_i - \hat{\pi}_i. \end{aligned}$$

$\hat{R}$ is the mean of the ℓ_i, so its derivative is the mean of theirs:

$$\frac{d\hat{R}}{d\theta} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{\pi}_i).$$

[a] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-log-loss

Theorem 3.1 (The slope of the soft rule's log loss)**.** *The mean log loss*[a] $\hat{R}$ *of the soft 0-or-1 rule has derivative*

$$\frac{d\hat{R}}{d\theta} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{\pi}_i)$$

with respect to its cutoff θ*.*

[a] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-log-loss

Proof

Proof. By Solution 3.2. The rule is a logistic regression[a] whose coefficient on x is fixed at 1 and whose intercept is $-\theta$, so the formula is also the negative of the intercept component of lds's gradient of the log loss[b], divided by n to give the mean. □

[a] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-logistic-regression
[b] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#thm-logistic-gradient

Example 3.1 (The slope at a cutoff of 5)**.**

```
x_np, y_np = ink_01.numpy(), is_one.numpy()
p_np = 1 / (1 + np.exp(-(x_np - 5)))
share_of_ones = np.mean(y_np)
mean_prob_at_5 = np.mean(p_np)
slope_by_hand = share_of_ones - mean_prob_at_5
print(share_of_ones, mean_prob_at_5, slope_by_hand)
```

```
0.5055555555555555 0.5473628863066621 -0.04180733075110654
```

By Theorem 3.1, the slope is the share of 1s among the images minus the average probability the rule gives them. The 1s make up a share of 0.506 of the 360 images, and at $\theta = 5$ the rule's average probability of a 1 is 0.547. So the slope at $\theta = 5$ is -0.0418. It is negative, so raising the cutoff from 5 lowers the log loss[a]. The slope is 0 at the cutoff where the average probability equals the share of 1s.

[a] https://morrison-lab.github.io/lds/chapters/logistic-regression.html#def-log-loss

3.3 The slope by autograd

Exercise 3.3 (Asking PyTorch for the slope)**.** Write a function that computes $\hat{R}(\theta)$ with PyTorch operations. Make $\theta = 5$ a tensor created with `requires_grad=True`, compute $\hat{R}(5)$, and pass both to `torch.autograd.grad`[a].

[a] https://docs.pytorch.org/docs/stable/generated/torch.autograd.grad.html

Solution

Solution 3.3.

```
def mean_log_loss(theta):
    """Mean log loss of the soft 0-or-1 rule with cutoff `theta`."""
    p = torch.sigmoid(ink_01 - theta)
    return -torch.mean(is_one * torch.log(p) + (1 - is_one) * torch.log(1 - p))

theta = torch.tensor(5.0, dtype=torch.float64, requires_grad=True)
loss = mean_log_loss(theta)
(slope_by_autograd,) = torch.autograd.grad(loss, theta)
print(loss)
```

```
tensor(0.0853, dtype=torch.float64, grad_fn=<NegBackward0>)
```

```
print(slope_by_autograd)
```

```
tensor(-0.0418, dtype=torch.float64)
```

`torch.autograd.grad` returns -0.0418. The code for $\hat{R}$ says nothing about its derivative. Because `theta` was made with `requires_grad=True`, PyTorch recorded each operation that turned it into `loss` (the subtraction `ink_01 - theta`, the logistic function, and each operation after them, down to the mean and its change of sign), and `torch.autograd.grad` applied the chain rule to those operations, one at a time. The `grad_fn` that `loss` prints is the record of its last operation.

Exercise 3.4 (Checking PyTorch's slope)**.** How far is the slope from Exercise 3.3 from the slope computed by hand in Example 3.1?

Solution

Solution 3.4.

```
slope_gap = abs(slope_by_autograd.item() - slope_by_hand)
print(slope_by_autograd.item(), slope_by_hand, slope_gap)
```

```
-0.04180733075110652 -0.04180733075110654 2.0816681711721685e-17
```

The two slopes differ by 2.1e-17, a floating-point rounding error: both are -0.0418 to four decimal places.

Definition 3.1 (Automatic differentiation (autodiff))**. Automatic differentiation**, also called **autodiff**, computes the derivatives of a function that is given as code, by applying the chain rule to the elementary operations the code performs as it runs [1], [3, Sec. 6.5].
PyTorch implements it in its `torch.autograd` module. The result is the exact derivative of the operations the code performs, up to floating-point rounding. Automatic differentiation is neither symbolic differentiation, which manipulates a formula for the function, nor numerical differentiation, which approximates a derivative from the function's values at nearby points. lds's backpropagation[a] chapter derives by hand the derivatives that autodiff computes for a neural network.

[a] https://morrison-lab.github.io/lds/chapters/backpropagation.html

Example 3.2 (The cutoff's slope by autodiff)**.** Solution 3.3 is automatic differentiation: PyTorch was given only the code for $\hat{R}$, and returned its derivative at $\theta = 5$, which matches the formula of Theorem 3.1 to within 2.1e-17, as Solution 3.4 found.

3.4 Gradient descent with autograd

Exercise 3.5 (Finding the cutoff by gradient descent)**.** Run 100 steps of gradient descent[a] on $\hat{R}$, starting from $\theta = 0$ with learning rate $\alpha = 4$, with each step's derivative from `torch.autograd.grad`. Where does θ end up?

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#the-algorithm

Solution

Solution 3.5.

```
learning_rate = 4.0
theta = torch.tensor(0.0, dtype=torch.float64, requires_grad=True)
for step in range(100):
    (slope,) = torch.autograd.grad(mean_log_loss(theta), theta)
    with torch.no_grad():
        theta -= learning_rate * slope
theta_descent = theta.item()
(final_slope,) = torch.autograd.grad(mean_log_loss(theta), theta)
print(theta_descent, final_slope.item())
```

```
6.59960743783184 -6.833215346312323e-06
```

Gradient descent[a] ends at $\theta = 6.5996$, where the slope is -6.8e-06.
The update runs inside `torch.no_grad()`, which tells PyTorch not to record it: the step is a change to θ, not part of the loss. Without it, PyTorch would record the in-place subtraction on `theta`, and it raises an error for a recorded in-place change to a tensor that was created with `requires_grad=True`.

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#the-algorithm

Exercise 3.6 (Gradient descent and the grid)**.** How far is the cutoff from Exercise 3.5 from the lowest point on the grid of cutoffs in the machine learning review?

Solution

Solution 3.6.

```
cutoffs = np.arange(0, 16.005, 0.01)
grid_step = cutoffs[1] - cutoffs[0]
with torch.no_grad():
    grid_loss = [mean_log_loss(torch.tensor(c)).item() for c in cutoffs]
theta_grid = cutoffs[np.argmin(grid_loss)]
gap = abs(theta_descent - theta_grid)
print(theta_grid, gap)
```

```
6.6000000000000005 0.00039256216816063727
```

The grid's lowest point is at 6.60, the cutoff found in the review's solution. It differs from the cutoff found by gradient descent[a] by 0.0004, less than the grid's spacing of 0.01.

[a] https://morrison-lab.github.io/lds/chapters/gradient-descent.html#the-algorithm

References

[1] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in neural information processing systems*, 2019. Available: https://arxiv.org/abs/1912.01703

[2] E. Alpaydin and C. Kaynak, "Optical recognition of handwritten digits." UCI Machine Learning Repository, 1998. doi: 10.24432/C50P49[4].

[3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT Press, 2016. Available: https://www.deeplearningbook.org/

[4] https://doi.org/10.24432/C50P49